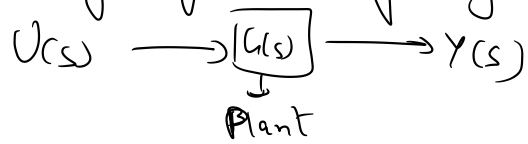
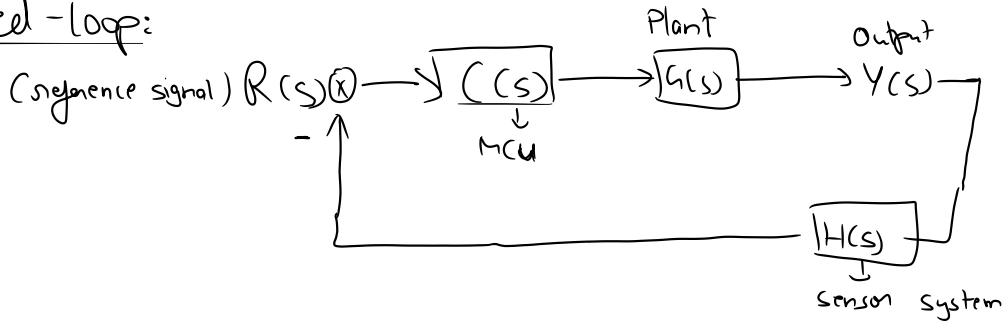


Plant: The physical system whose behavior we are trying to control.

Open-loop: Blind to actual output. Gives control signal based on fixed transfer function for system, to achieve desired output



Closed-loop:



Solving for $\frac{Y(s)}{R(s)} = \frac{G(s)C(s)}{1 + G(s)C(s)H(s)}$

Motor plant is a second order system, as it has two major energy storing elements: Motor's inertia, motor coils inductance

$$V(s) = I(s)(Ls + R) + E_b(s) ; \quad \begin{aligned} & \tau = \omega(s)(Js + b) \quad \begin{aligned} & \nearrow \text{motor shaft/load inertia} \\ & \searrow \text{damping/frictional constant} \end{aligned} \\ & \hookrightarrow \text{Torque} = K_t \cdot I(s) \end{aligned}$$

Also, $E_b = K_e \cdot \omega(s)$

($\because E_b \propto \frac{d\phi}{dt} \propto \omega$)

$\therefore I(s) = \frac{\omega(s)}{K_t} (Js + b)$

$\Rightarrow V(s) = \frac{\omega(s)}{K_t} (Js + b)(Ls + R) + K_e \omega(s)$

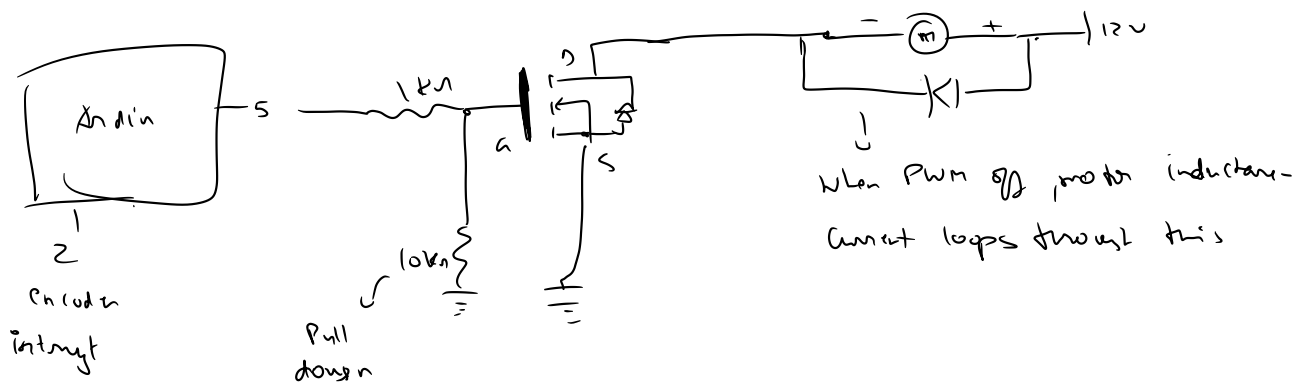
$\Rightarrow \frac{\omega(s)}{V(s)} = G(s) = \frac{K_t}{(Js + b)(Ls + R) + K_t K_e} \quad \Rightarrow 2 \text{ poles, No zeros}$

System Identification: Refer to git -repo

Need for RC filter to remove quantization error sample time = 0.02s
↳ To filter $\frac{1}{\tau_{st}}$, cut off freq = $\omega_c = \frac{1}{\tau} = 50 \text{ rad/s}$

$$f = 8 \text{ Hz}$$

The encoder is well below this



To be even more accurate → accounting for voltage sag, we can measure voltage across motor terminals using voltage divider circuit

Step response: $0 \rightarrow$ whatever you set, with ideally infinite slope

Rise time: Time to go from 10% to 90% of steady state goal

Faster rise time $\rightarrow$ higher gain $\rightarrow$ but physical non-linearities (like +12V limit)

Overshoot: how far the system pushed past the goal,
like if sys had high inertia, and we used high gain, the system may overshoot if the gain isn't stopped in time.

Settling time: Time it takes for sys to stay within 2% of target.

Steady state error: Due to physical deadzones, like motor friction, normal P will always have SSE because eventually the applied effort = deadzone effort requirement, and then nothing changes, $\therefore$ need Integral term.

Poles, stability, oscillations: (Explainable) $\zeta = -\dot{\omega}_n$, $\omega = \omega_n \sqrt{1 - \zeta^2}$

$\downarrow$
Need to ensure gain < loss than friction/resistance
 $\downarrow$
push < damping $\Rightarrow$ stability

PID: P: current error response. pushes toward target but is blind to SSE

I: Accumulated error history $\rightarrow$ deals with SSE

But, integration $\rightarrow \frac{1}{s} \rightarrow$ delayed $\rightarrow 90^\circ$ phase lag

In negative feedback loop 180° (inversion) $+ 90^\circ$ (motor) $+ 90^\circ$ ($\frac{1}{s}$) $= 360^\circ = 0^\circ$

g lag, $\rightarrow$ basically created positive feedback, so if overshoot happens, the lagging logic will positively push for more error

By adding 'D' we remove $90^\circ \rightarrow \therefore 270^\circ$ lag

D: Predicts where the system is going based on rate of change of error.

Acts like a shock absorber (If P is pushing too hard and error decreasing very fast, reduces gain to prevent overshoot)

(os)

Tuning: P: Inc till to reach goal in acceptable time (wise time)

I: slowly inc to reduce SSE, carefully to not inc oscillation due to lag.

D: To damp till overshoot is reduced.

Ziegler-Nichols method: Made for predetermined system needs, not useful if we want to set our requirements

Actuator/Supply Saturation: Gain enough for something absurd like 100 V, but physically limited to ~~2~~ 24V

Rate limit: Speed of change is limited, supply cannot instantly jump from 5 to 25 Amps, the wire itself has parasitic inductances.

Mechanical / electrical inertia

∴ If controller wants faster change than possible the actuator/motor will lag behind command, creating unaccounted for phase delay.

↳ Creates non-linearity, suppose I input a high freq sin input for my motor to trace, due to rate limit, the output shaft isn't able to track it effectively → we get something like a triangular wave
↳ Use a 'trajectory planner / slew rate limiter' to combat this → need to study

Integral wind: The accumulated term gets too massive and continues pushing output beyond goal, even though error is now opp, but k_p is too weak to counter the massive I term.

Ex: If input saturates, so output grows slowly, but I term keeps accumulating this error.

Standard fixes: (clamping: If output > saturation limit, freeze the Integral term.

Back-calc (Dynamic unwinding): ?? Didn't understand

Derivative Kick: (Relates to quantization error and massive slopes in it due to digital steps)

$$e(k) = \underset{\substack{\downarrow \\ \text{reference} \\ \downarrow \\ \text{digital} \rightarrow \text{spiky} \rightarrow \text{xx bad}}}{r(k)} - \underset{\substack{\downarrow \\ \text{output} \rightarrow \text{analog} \rightarrow \text{continuous} \rightarrow \text{all good}}}{y(k)}$$

$$\frac{de}{dt} = \underbrace{\frac{dy}{dt}}_{\text{Problematic}} - \frac{dy}{dt}, \text{ fix is to set } \frac{dn}{dt} = 0$$

And just use $\frac{dy}{dt}$, and absorb the effect into K_d

BUT, $y \rightarrow$ is also through some digital encoder/sensor

The actual motor shaft slowly speeds or slows down?

low freq waves

$\therefore$ Use a low pass filter $\therefore$ Final term = $\frac{K_d \cdot s}{s+1}$

We want K_d to see only these

$$s \downarrow \quad T \cdot s \downarrow \quad \text{Final} = K_d \cdot s$$

$$s \uparrow \quad T \cdot s \uparrow \quad T \cdot s + 1 \approx T \cdot s \therefore \text{Final} = \frac{K_d}{T} = \text{capped} \rightarrow \text{doesn't "kick"}$$

Discretization: Need to model simulation tuning considering sampling time of hardware.

Hardware sampling time can be optimized by reducing bloat code like serial prints, delay blocks.

Feedforward: Output = $U_{pid} + U_{\text{feedforward}}$

Since PID needs error term to function, once it reaches goal, it gives up $\rightarrow$ external known forces take over, again cause error, eventually PID takes back control, this keeps happening.

So if we know hardcoded physical need to ensure stability we can feed it directly, and PID can do the clean up to ensure it stays stable there.

Cascade: basically splitting one massive transfer function into multiple blocks that can be tuned more effectively as each block has reduced physical complexity to consider.

Also, can implement saturation limits b/w blocks by adding ops.

State space: Allows tracking of internal system variables
Energy holders

$$\dot{x} = Ax + Bu$$

$$y = Cx + Du$$

State space advantage:

1) MIMO: Multiple inputs in U vector to give multiple Y outputs,
Allow for multiple feature control, ex: In a drone I have input
voltage $\rightarrow$ output roll, pitch, yaw, altitude

$\therefore$ 4 transfer func.

But each motor affects the overall output differently to
total $4 \times 4 = 16$ transfer func.

2) Transfer assume Initial conditions zero, state space doesn't
allowing for control in scenarios with initial conditions like
drone recovery when mid-air

LQR: Algorithm that generates perfect controller based on
a certain "budget" and goals, we define

Goal is to make $J \rightarrow 0$, $J = \int_0^{\infty} (x^T Q x + u^T R u) \cdot dt$

i) $x^T Q x$: x is state vector, Q is defined by us: describes how much
we hate error $Q \uparrow \Rightarrow$ hate $\uparrow$

ii) $u^T R u$: u is input vector, R is how much we hate using input energy
 $R \uparrow \Rightarrow$ Hate using more power $\uparrow$

iii) $\int_0^{\infty}$: We are trying to minimise this over the entire machines
lifetime, not shortsighted benefit

(ARE) something $\rightarrow$ gives optimal gain matrix

$$u(t) = -Kx(t)$$

$\therefore$ Perfect input signal

Basically, we can set if our priority is absolute minimal energy or do we want to save how much we force our actuator to work or how much power were going to draw.

Real world problems: 1) LQR absolutely requires all the state variables info, might be practically hard to implement all of these.

Fix: We guess the missing state, using a 'Kalman filter' (??)

2) LQR ignore saturation limits $\rightarrow$ fix use MPC

3) LQR works linear time, A, B not func of time.

But real world $\rightarrow$ not always true

Fix: Separate A, B for different operating modes, within which we can approximate $\frac{dA}{dt} = 0 = \frac{dB}{dt}$

MPC: 1) Prediction horizon: How far ahead are predicting.

2) Control horizon: how far ahead do we find the steps to perform for.

3) Cost function: - -

4) constraints: Can specify actuator / sat limits here

5) Optimizer: Finds and gives best control algo.

Overall, to assume short term perfection, it doesn't ruin long term

Problem: Unlike LQR where we needed to do our time 'k' calc, here this complex algo runs again and again

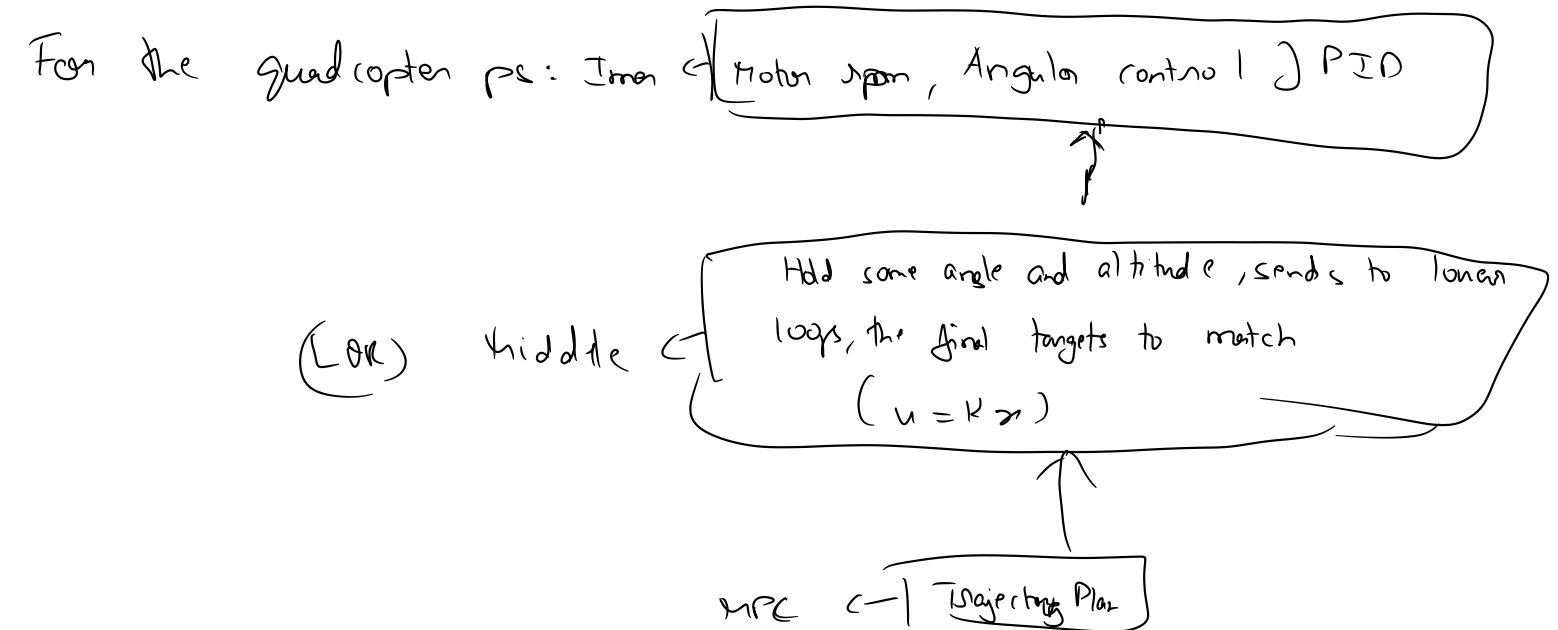
What to use and when

1) PID: When a single input output relationship is to be controlled, at fast frequencies.
Single wheel speed control using a motor

2) LQR: For MIMO systems, leveraging state vectors, for systems where output affects all others. E.g.: drone roll, pitch, yaw, altitude control.

3) MPC: For high level path planning / obstacle avoidance.

Note: LQR, MPC vulnerable to variation of hardware, as A, B, C, D are fixed, but hardware characteristics might change



Task A: step of 2nd order output goal
under 10% overshoot
settling in 0.5s
Zero SSE

Plant: $G(s) = \frac{W(s)}{V(s)} = \frac{1.5}{(0.35s+1)}$

Solving for $\frac{W(s)}{R(s)}$



$$T(s) = \frac{W(s)}{R(s)} = \frac{(s)G(s)}{1 + (s)G(s)}$$

Also, since $C(s) = \frac{K_i}{s} + K_p$ (Don't need K_d , as later we'll see how $K_p \rightarrow \zeta \rightarrow$ controls damping and thus overshoot) (Why even use K_d then?)

$$\therefore T(s) = \frac{\left(\frac{K_p s + K_i}{s}\right) \left(\frac{1.5}{0.35s+1}\right)}{s + \left(\frac{K_p s + K_i}{s}\right) \left(\frac{1.5}{0.35s+1}\right)} \Rightarrow \frac{(K_p s + K_i) (3/2)}{s(0.35s+1) + (1.5)(K_p s + K_i)}$$

$$\Rightarrow \frac{(1.5)(K_p s + K_i)}{(0.35s^2 + s(1.4K_p) + 1.5K_i)}$$

$\zeta \propto \frac{K_p}{\sqrt{K_i}}$ This is a 1st order plant, so, but in 2nd

order plants this is invalid

$\hookrightarrow$ To avoid derivative

Kick wherever possible
since K_d not needed, don't use

$\Rightarrow$ By second order standard

eg, $\omega_n^2 = \frac{1.5K_i}{0.35}$, $2\zeta\omega_n = \frac{1 + 1.5K_p}{0.35}$

$T_s = \frac{4}{\zeta\omega_n} \leq 0.5 \therefore \zeta\omega_n \geq 8$, let $\zeta\omega_n = 10$

$M_p < 10\% \Rightarrow \zeta = 0.6$, further the extra s in the numerator acts like a derivative kick on the input signal, allowing for faster rise but also overshoot, $\therefore$ selecting $\zeta = 1$ instead of 0.6 for more damping.

$\therefore K_i = 23.33$, $K_p = 4$ Initial gains

Task B: Plant is limited to rotate at 18 rad/s

so the crazy oscillation due to saturation doesn't occur

$$\omega_{\text{final steady state}} = \lim_{s \rightarrow 0} s \cdot F(s) = s \cdot \frac{(1.5) \cdot V(s)}{(0.35s+1)}$$

$$= \frac{(1.5)(12)}{(0.35s+1)} \cdot \frac{1}{s}$$

$$\Rightarrow \underline{\underline{18 \text{ rad/s}}}$$

$\therefore$ Even though I is massive, nothing comes out of it.